

Exp : 1

Date:

BASIC LINUX COMMANDS

AIM: To understand the Linux environment by exploring the login process, terminal interface, and hierarchical file system structure, and to demonstrate proficiency in using basic commands: ls, pwd, cd, clear, whoami, date, and cal.

PROCEDURE:

1. **System Login:** Start the Linux machine. At the login prompt, enter your Username and Password.
2. **Open Terminal:** Once the GUI (Graphical User Interface) loads, press Ctrl + Alt + T or search for "Terminal" in the application menu to open the Command Line Interface (CLI).
3. **Identify User:** Type "whoami" and press Enter to confirm which user account is currently active.
4. **Check Location:** Type "pwd" to see your current position within the Linux directory tree (the "Home" directory by default).
5. **List Files:** Type "ls" to view all folders and files available in your current location.
6. **Navigate Directories:** Type "cd Documents" to move into the Documents folder. Type "cd .." to move back up one level.
7. **Check Time & Date:** Type "date" to view the current system time and "cal" to view the calendar for the current month.
8. **Clean Workspace:** Type "clear" to remove all previous command outputs and reset the terminal view.

LINUX COMMANDS:

Command 1: whoami – To see the current user name

Syntax: \$ whoami

Output:

```
[student@ADS-20s-iMac music % whoami  
student  
student@ADS-20s-iMac music % █
```

Command 2: pwd – To check present working directory

Syntax: \$ pwd

Output:

```
[student@ADS-20s-iMac ~ % pwd  
/Users/student
```

Command 3: ls – To list the files in the directory

Syntax: \$ ls

Output:

```
[student@ADS-20s-iMac ~ % ls  
#include <stdio.h> Downloads Public  
#include<stdio.h> Library sri1.c  
Applications Movies test  
c.2 Music workshop  
data structure myenv yuvan  
Desktop output  
Documents Pictures  
[student@ADS-20s-iMac ~ %
```

Command 4: cd – To change the directory

Syntax: \$ cd [directory]

Output:

```
[student@ADS-20s-iMac ~ % cd music  
[student@ADS-20s-iMac music %
```

Command 5: date – To the present date

Syntax: \$ date

Output:

```
[student@ADS-20s-iMac music % date  
Tue Feb 3 15:03:59 IST 2026
```

Command 6: cal – To see the calendar

Syntax: \$ cal

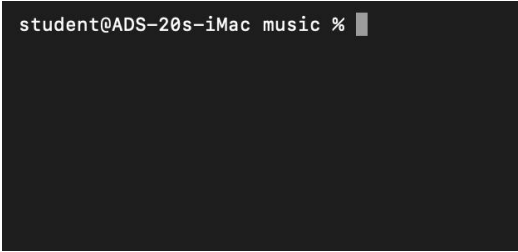
Output:

```
[student@ADS-20s-iMac music % cal  
February 2026  
Su Mo Tu We Th Fr Sa  
1 2 3 4 5 6 7  
8 9 10 11 12 13 14  
15 16 17 18 19 20 21  
22 23 24 25 26 27 28
```

Command 7: clear – To clear the linux terminal

Syntax: \$ clear

Output:

A screenshot of a Linux terminal window. The prompt is 'student@ADS-20s-iMac music %'. The command 'clear' has been entered, and the terminal screen is now blank, demonstrating the effect of the 'clear' command.

```
student@ADS-20s-iMac music %
```

R1(5)	R2(5)	R3(5)	R4(5)	Total (20)

RESULT:

The Linux environment was successfully explored. The login process was completed, and fundamental commands were executed to navigate the file system and retrieve system information.

Exp : 2

Date:

FILE DIRECTORY HANDLING COMMANDS

AIM: To Demonstrate Proficiency In Linux File and Directory File And Directory Handling Commands Like mkdir, rmdir, touch, cat, cp, mv, rm, head, tail, more, less.

PROCEDURE:

1. Launch Terminal: Open the command line using Ctrl + Alt + T.
2. Create/Delete Directory: Use mkdir myfolder to create a directory and rmdir myfolder to remove it.
3. File Creation: Use touch file1.txt to create an empty file.
4. View & Edit Content: Use cat to display file content, head for the first 10 lines, and tail for the last 10 lines.
5. File Operations: Copy files using cp, rename or move them using mv, and delete them using rm.
6. Advanced Viewing: Use more or less to scroll through large files page by page.

LINUX COMMANDS:

Command 1 : mkdir (Used to create a new directory or folder)

Syntax: \$ mkdir

Output:

```
student@ADS-20s-iMac ~ % mkdir rro
student@ADS-20s-iMac ~ % ls
#include <stdio.h>  Applications      data structure  Documents      Library        Music          output          Public
#include <stdio.h>  c.2              Desktop        Downloads      Movies         myenv          Pictures        rro
```

Command 2 : rmdir (Used to remove an empty directory)

Syntax: \$ rmdir

Output:

```
kprais@ADS-10s-iMac ~ % mkdir s
kprais@ADS-10s-iMac ~ % rmdir s
kprais@ADS-10s-iMac ~ % ls
Applications      Library          Public           ex03.html       newex03.html    notes.js
Desktop           Movies          SUREKA.html     exreg.html      news.ejs        nova
Documents         Music           Sites            file1.txt       node.js         onlineex2.html
Downloads         Pictures        app.js          index.ejs       notes-app-backend reg.html
kprais@ADS-10s-iMac ~ %
```

Command 3 : touch (Used to create a new empty file or update timestamps)

Syntax: \$ touch

Output:

```
[student@ADS-20s-iMac ~ % touch linux.txt
```

Command 4 : cat (Used to display, combine, or create file content)

Syntax: \$ cat

Output:

```
zsh: suspended cat > linux.txt
[student@ADS-20s-iMac ~ % cat linux.txt
hey there!!
[student@ADS-20s-iMac ~ % cat >> linux.txt
hello world!!

^Z
zsh: suspended cat >> linux.txt
[student@ADS-20s-iMac ~ % cat linux.txt
hey there!!
hello world!!

student@ADS-20s-iMac ~ % █
```

Command 5 : cp (Used to copy files or directories from one place to another)

Syntax: \$ cp

Output:

```
[student@ADS-20s-iMac ~ % cp aiml.txt linux.txt
[student@ADS-20s-iMac ~ % cat linux.txt
i like pizza ! its delicious!!
```

Command 6: mv (Used to move or rename files and directories)

Syntax: \$ mv

Output:

```
[student@ADS-20s-iMac ~ % mv linux.txt aiml.txt
[student@ADS-20s-iMac ~ % cat aiml.txt
i like pizza ! its delicious!!

z

[student@ADS-20s-iMac ~ % cat linux.txt
cat: linux.txt: No such file or directory
student@ADS-20s-iMac ~ % █
```

Command 7 : rm (Used to remove or delete files)

Syntax: \$ rm

Output:

```
student@ADS-1s-iMac ~ % rm aiml
student@ADS-1s-iMac ~ %
```

Command 8 : head (Used to view the beginning of a file)

Syntax: \$ head

Output:

```
kpraid@ADS-10s-iMac ~ % head ml.txt
welcome to my folder
kpraid@ADS-10s-iMac ~ %
```

Command 9 : tail (Used to view the end of a file)

Syntax: \$ tail

Output:

```
kpraid@ADS-10s-iMac ~ % tail ml.txt
welcome to my folder
kpraid@ADS-10s-iMac ~ %
```

Command 10 : more (Used to view file content one screen at a time)

Syntax: \$ more

Output:

```
kpraid@ADS-10s-iMac ~ % more nova.txt
hello how was your day
cat nova.txt
nova.txt (END)
```

Command 11 : less (Used to view file content with advanced backward/forward navigation)

Syntax: \$ less

Output:

```
~$ less linuxcmd
~$
linux is useful
and it is easier
to work
it is helpful
it shows what
computer backend does
linuxcmd (END)
```

R1(5)	R2(5)	R3(5)	R4(5)	Total (20)

RESULT:

The Linux environment was successfully explored. The login process was completed, and fundamental commands were executed to navigate the file system and retrieve system information.

Exp : 3

Date:

FILE PERMISSIONS AND OWNERSHIP

AIM:

To Use of chmod, chown, umask, symbolic and numeric modes of permissions.

PROCEDURE:

1. Open the Linux terminal.
2. Create a file in the working directory.
3. Check the file permissions using `ls -l`.
4. Modify permissions using `chmod` with symbolic mode.
5. Modify permissions using `chmod` with numeric mode.
6. Change file ownership using `chown`.
7. Display the default permission settings using `umask`.
8. Change the default permission using `umask`.
9. Verify the changes using `ls -l`.

LINUX COMMANDS

Command 1 : Check Current Permissions

Syntax : `ls -l`

Output :

```
monu@Victus-1:~/exp3$ ls -lh
total 8.0K
drwxr-xr-x 2 monu monu 4.0K Mar  4 13:55 demo1
drwxr-xr-x 2 monu monu 4.0K Mar  4 13:53 demo2
monu@Victus-1:~/exp3$ |
```

Command 2 : Basic Command Remove Permission

Syntax : `chmod -[options] permissions filename`

Output :

```
monu@Victus-1:~/exp3$ ls -lh
total 8.0K
drwxr-xr-x 2 monu monu 4.0K Mar  4 13:55 demo1
drwxr-xr-x 2 monu monu 4.0K Mar  4 13:53 demo2
monu@Victus-1:~/exp3$ chmod -w demo1
monu@Victus-1:~/exp3$ ls -lh
total 8.0K
dr-xr-xr-x 2 monu monu 4.0K Mar  4 13:55 demo1
drwxr-xr-x 2 monu monu 4.0K Mar  4 13:53 demo2
monu@Victus-1:~/exp3$
```

Command 3 : Add Permission

Syntax : `chmod +[options] permissions filename`

Output :

```

monu@Victus-1:~/exp3$ ls -lh
total 8.0K
dr-xr-xr-x 2 monu monu 4.0K Mar  4 13:55 demo1
drwxr-xr-x 2 monu monu 4.0K Mar  4 13:53 demo2
monu@Victus-1:~/exp3$ chmod g+w demo1
monu@Victus-1:~/exp3$ ls -lh
total 8.0K
dr-xrwxr-x 2 monu monu 4.0K Mar  4 13:55 demo1
drwxr-xr-x 2 monu monu 4.0K Mar  4 13:53 demo2
monu@Victus-1:~/exp3$

```

Command 4 : Set Specific Permission

Syntax : chmod [options]=[option] permissions

Output :

```

monu@Victus-1:~/exp3$ ls -lh
total 8.0K
dr-xrwxr-x 2 monu monu 4.0K Mar  4 13:55 demo1
drwxr-xr-x 2 monu monu 4.0K Mar  4 13:53 demo2
monu@Victus-1:~/exp3$ chmod g=r demo1
monu@Victus-1:~/exp3$ ls -lh
total 8.0K
dr-xr--r-x 2 monu monu 4.0K Mar  4 13:55 demo1
drwxr-xr-x 2 monu monu 4.0K Mar  4 13:53 demo2
monu@Victus-1:~/exp3$

```

Command 5 : Multiple Changes

Syntax : chmod [options][option] permissions filename

Output :

```

monu@Victus-1:~/exp3/demo1$ ls -lh
total 0
-rw-r--r-- 1 monu monu 0 Mar  4 13:55 demo1
-rw-r--r-- 1 monu monu 0 Mar  4 13:55 demo2
-rw--w-r-- 1 monu monu 0 Mar  4 14:18 demo3
monu@Victus-1:~/exp3/demo1$ chmod g+w+x demo1
monu@Victus-1:~/exp3/demo1$ ls -lh
total 0
-rw-rwxr-- 1 monu monu 0 Mar  4 13:55 demo1
-rw-r--r-- 1 monu monu 0 Mar  4 13:55 demo2
-rw--w-r-- 1 monu monu 0 Mar  4 14:18 demo3
monu@Victus-1:~/exp3/demo1$

```

Command 6 : Using Numeric (Octal) Mode

Syntax : chmod Octal permissions filename

Output :

```

monu@Victus-1:~/exp3$ ls -lh
total 8.0K
dr-xr--r-x 2 monu monu 4.0K Mar  4 13:55 demo1
drwxr-xr-x 2 monu monu 4.0K Mar  4 13:53 demo2
monu@Victus-1:~/exp3$ chmod 755 demo1
monu@Victus-1:~/exp3$ ls -lh
total 8.0K
drwxr-xr-x 2 monu monu 4.0K Mar  4 13:55 demo1
drwxr-xr-x 2 monu monu 4.0K Mar  4 13:53 demo2
monu@Victus-1:~/exp3$

```

Command 7 : Using Umask

Syntax : umask folder

Output :

```

monu@Victus-1:~/exp3/demo1$ ls -lh
total 0
-rw-r--r-- 1 monu monu 0 Mar  4 13:55 demo1
-rw-r--r-- 1 monu monu 0 Mar  4 13:55 demo2
monu@Victus-1:~/exp3/demo1$ umask g+wx
monu@Victus-1:~/exp3/demo1$ touch demo3
monu@Victus-1:~/exp3/demo1$ ls -lh
total 0
-rw-r--r-- 1 monu monu 0 Mar  4 13:55 demo1
-rw-r--r-- 1 monu monu 0 Mar  4 13:55 demo2
-rw--w-r-- 1 monu monu 0 Mar  4 14:18 demo3
monu@Victus-1:~/exp3/demo1$

```

R1(5)	R2(5)	R3(5)	R4(5)	Total (20)

Result :

All the permission and ownership commands as been successfully executed on Linux environment.

Exp : 4
Date :

LINUX UTILITIES FILE SEARCHING AND FILTERING

AIM:

To understand and perform file searching and data filtering in Linux using utilities like find, locate, and grep, and to process and analyse text using commands such as sort, uniq, cut, wc, diff, and cmp.

PROCEDURE:

1. Open the terminal and navigate to the working directory.
2. Use find and locate to search for files.
3. Use grep to identify patterns within files.
4. Apply sort and uniq to organize and filter the data.
5. Use cut to extract specific fields.
6. Use wc to count lines, words, and characters.
7. Use diff and cmp to compare files.
8. Observe and analyse the outputs.

LINUX COMMANDS

Command 1 : To find the location of a file.

Syntax : find <File_Name>

Output :

```
monu@Victus-1:~$ find exp3
exp3
exp3/demo2
exp3/demo1
exp3/demo1/demo2
exp3/demo1/demo1
exp3/demo1/demo3
monu@Victus-1:~$ |
```

Command 2 : Search text patterns.

Syntax : grep "pattern" filename

Output :

```
monu@Victus-1:~/exp3/demo2$ grep "Hi" word
Hi This is my log for the linux command
monu@Victus-1:~/exp3/demo2$ |
```

Command 3 : Sort lines in a file.

Syntax : sort filename

Output :

```
monu@Victus-1:~/exp3/demo2$ sort word
And on this i am testing various linux commands
Hi This is my log for the linux command
monu@Victus-1:~/exp3/demo2$ |
```

Command 4 : Remove duplicate adjacent lines.

Syntax : uniq filename

Output :

```
monu@Victus-1:~/exp3/demo2$ cat >>word
AA
AA
AA
BB
BB
B
BB
cde
^Z
[2]+  Stopped                  cat >> word
monu@Victus-1:~/exp3/demo2$ uniq word
Hi This is my log for the linux command
And on this i am testing various linux commands
AA
BB
B
BB
cde
monu@Victus-1:~/exp3/demo2$ |
```

Command 5 : Extract specific columns/fields from text.

Syntax : cut [option] filename

Output :

```
monu@Victus-1:~/exp3/demo2$ cut -c 1-4 word
Hi T
And
AA
AA
AA
BB
BB
B
BB
cde
monu@Victus-1:~/exp3/demo2$
```

Command 6 : Count lines, words, and characters.

Syntax : wc filename

Output :

```
monu@Victus-1:~/exp3/demo2$ wc word
 10  26 115 word
monu@Victus-1:~/exp3/demo2$ wc -l word
10 word
monu@Victus-1:~/exp3/demo2$ wc -w word
26 word
monu@Victus-1:~/exp3/demo2$
```

Command 7 : Compare two files line by line.

Syntax : diff file1.txt file2.txt

Output :

```

monu@Victus-1:~/exp3/demo2$ diff word word2
1,2c1
< Hi This is my log for the linux command
< And on this i am testing various linux commands
---
> This is file 2 and this is used to check duplicates
4,8d2
< AA
< AA
< BB
< BB
< B
9a4,6
> AB
> AA
> AAAA
monu@Victus-1:~/exp3/demo2$

```

Command 8 : Compare two files byte by byte.

Syntax : cmp file1 file2

Output :

```

monu@Victus-1:~/exp3/demo2$ cmp word word2
word word2 differ: byte 1, line 1
monu@Victus-1:~/exp3/demo2$

```

R1(5)	R2(5)	R3(5)	R4(5)	Total (20)

RESULT:

All the linux utilities have been tested successfully and the command executed.

Exp : 5

Date:

PROCESS MANAGEMENT AND SYSTEM MONITORING

AIM:

To understand the Linux environment by studying the login process, using the terminal, and working with system management commands such as ps, top, kill, nice, df, du, uptime, free, uname, and who.

PROCEDURE:

1. Open the Linux terminal and log in.
2. Use who and uname to check user and system information.
3. Check system status using uptime and free.
4. View disk usage with df and du.
5. Monitor processes using ps and top.
6. Use nice to change process priority and kill to terminate a process.
7. Observe the outputs.

LINUX COMMANDS

Command 1 : Shows running processes on your system.

Syntax : ps

Output :

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ ps
ps -ef
ps aux
  PID TTY          TIME CMD
  409 pts/2    00:00:00 bash
  545 pts/2    00:00:00 ps
UID          PID    PPID  C STIME TTY          TIME CMD
root         1        0  0 16:29 ?        00:00:00 /sbin/init
root         2        1  0 16:29 ?        00:00:00 /init
root         6        2  0 16:29 ?        00:00:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root        42        1  0 16:29 ?        00:00:00 /usr/lib/systemd/systemd-journald
root        89        1  0 16:29 ?        00:00:00 /usr/lib/systemd/systemd-udevd
systemd+   104        1  0 16:29 ?        00:00:00 /usr/lib/systemd/systemd-resolved
systemd+   105        1  0 16:29 ?        00:00:00 /usr/lib/systemd/systemd-timesyncd
root       158        1  0 16:29 ?        00:00:00 /usr/sbin/cron -f -p
message+   159        1  0 16:29 ?        00:00:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activatio
root       166        1  0 16:29 ?        00:00:00 /usr/lib/systemd/systemd-logind
root       171        1  0 16:29 hvc0    00:00:00 /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 vt220
syslog     176        1  0 16:29 ?        00:00:00 /usr/sbin/rsyslogd -n -iNONE
root       189        1  0 16:29 tty1    00:00:00 /sbin/agetty -o -p -- \u --noclear - linux
root       196        1  0 16:29 ?        00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wai
root       286        2  0 16:29 pts/1   00:00:00 /bin/login -f
rithik     329        1  0 16:29 ?        00:00:00 /usr/lib/systemd/systemd --user
rithik     330       329  0 16:29 ?        00:00:00 (sd-pam)
rithik     355       286  0 16:29 pts/1   00:00:00 -bash
root       404        2  0 16:32 ?        00:00:00 /init
root       405       404  0 16:32 ?        00:00:00 /init
rithik     409       405  0 16:32 pts/2    00:00:00 -bash
rithik     546       409  0 16:51 pts/2    00:00:00 ps -ef
```

Command 2 : real-time process monitor

Syntax : top

Output :

```
rithik@DESKTOP-80QDGT4:~$ top
top - 16:27:53 up 2:21, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 23 total, 1 running, 22 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3778.2 total, 3197.1 free, 411.4 used, 255.6 buff/cache
MiB Swap: 1024.0 total, 1024.0 free, 0.0 used, 3366.8 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
  5923 rithik    20   0   9276   5504   3456 R   0.3   0.1   0:00.07 top
    1 root      20   0  22096  13056   9600 S   0.0   0.3   0:03.30 systemd
    2 root      20   0   3120   2048   1920 S   0.0   0.1   0:00.05 init-systemd(Ub
    6 root      20   0   3136   1792   1792 S   0.0   0.0   0:00.00 init
   176 root      20   0   4236   2560   2432 S   0.0   0.1   0:00.04 cron
   177 message+ 20   0   9848   5120   4352 S   0.0   0.1   0:00.81 dbus-daemon
   184 root      20   0  17964   8320   7552 S   0.0   0.2   0:00.27 systemd-logind
   188 root      20   0   3160   2048   1920 S   0.0   0.1   0:00.00 agetty
   208 root      20   0   3116   1792   1792 S   0.0   0.0   0:00.00 agetty
   215 root      20   0 107024  22272  13184 S   0.0   0.6   0:00.13 unattended-upgr
   383 root      20   0   3124    768    768 S   0.0   0.0   0:00.00 SessionLeader
   384 root      20   0   3140   1152   1024 S   0.0   0.0   0:00.46 Relay(385)
   385 rithik    20   0   7540   5888   4224 S   0.0   0.2   0:00.29 bash
   505 root      20   0   6688   4352   3712 S   0.0   0.1   0:00.00 login
   546 rithik    20   0  20012  10880   9216 S   0.0   0.3   0:00.20 systemd
   547 rithik    20   0  21152   3516   1792 S   0.0   0.1   0:00.00 (sd-pam)
   570 rithik    20   0   6056   4992   3456 S   0.0   0.1   0:00.02 bash
  3362 syslog    20   0  222508  5120   4480 S   0.0   0.1   0:00.10 rsyslogd
  4064 root      19  -1  66720  15244  14348 S   0.0   0.4   0:00.87 systemd-journal
  4117 systemd+  20   0   91028   7808   6912 S   0.0   0.2   0:00.35 systemd-timesyn
  4170 root      20   0  24988   6228   5076 S   0.0   0.2   0:01.23 systemd-udev
  4226 systemd+  20   0  21460  13056  10880 S   0.0   0.3   0:00.18 systemd-resolve
  4644 polkitd   20   0 308164   7808   7040 S   0.0   0.2   0:00.33 polkitd
```

Command 3: set process priority (CPU scheduling)

Syntax: nice

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ nice -n 10 sleep 5
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$
```

Command 4: Shows disk free (storage usage)

Syntax: df

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ df
Filesystem      1K-blocks    Used Available Use% Mounted on
none            1934448      0    1934448   0% /usr/lib/modules/6.6.87.2-microsoft-standard-WSL2
none            1934448      4    1934444   1% /mnt/wsl
drivers         249190396 154158060 95032336 62% /usr/lib/wsl/drivers
/dev/sdd        1055762868 1735640 1000323756 1% /
none            1934448      76    1934372   1% /mnt/wslg
none            1934448      0    1934448   0% /usr/lib/wsl/lib
rootfs          1929408     2720    1926688   1% /init
none            1934448      500    1933948   1% /run
none            1934448      0    1934448   0% /run/lock
none            1934448      0    1934448   0% /run/shm
none            1934448      76    1934372   1% /mnt/wslg/versions.txt
none            1934448      76    1934372   1% /mnt/wslg/doc
C:\             249190396 154158060 95032336 62% /mnt/c
tmpfs           386888      20     386868   1% /run/user/1000
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ |
```

Command 5: disk usage (files & folders)

Syntax:du

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ du -sh /home 52k
52K    /home
```

Command 6: how long system is running + load

Syntax: uptime

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ uptime
17:06:45 up 38 min, 1 user, load average: 0.04, 0.08, 0.04
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ |
```

Command 7: Shows RAM usage

Syntax:free

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ free
              total        used        free      shared    buff/cache   available
Mem:           3868896      395644      3391232         3568        157616        3473252
Swap:          1048576           0         1048576
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ |
```

Command 8: uname shows system information

Syntax: uname

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ uname
Linux
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ |
```

Command 9:who is logged into the system

Syntax:who

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ who
rithik pts/1      2026-04-28 17:03
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ |
```

Command 9:Kill the running task

Syntax:kill PID

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ kill 472
bash: kill: (472) - No such process
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ |
```

R1(5)	R2(5)	R3(5)	R4(5)	Total (20)

RESULT

The Linux environment was successfully studied, and various system commands were used to monitor processes, manage resources, and analyze system information effectively.

Exp : 6
Date:

ARCHIVING AND COMPRESSION UTILITIES

AIM:

To study and perform archiving and compression utilities in Linux using tar, gzip, gunzip, zip, unzip, and xz.

PROCEDURE:

1. Open the Linux terminal.
2. Create sample files and folders.
3. Use tar command to create archive files.
4. Compress files using gzip, zip, and xz.
5. Decompress files using gunzip, unzip, and unxz.
6. Extract archive files using tar.
7. Verify the compressed and extracted files.

LINUX COMMANDS

Command 1: tar - create and extract archive files

Syntax:

tar -cvf archive.tar file

tar -xvf archive.tar

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ touch file.txt
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ tar -cvf test.tar file.txt
file.txt
```

Command 2: gzip - compress file using gzip

Syntax:

gzip filename

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ gzip log.txt
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ ls
log.txt.gz
```

Command 3: gunzip - decompress gzip compressed file

Syntax:

gunzip filename.gz

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ gunzip log.txt.gz
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ ls
log.txt
```

Command 4: zip - create compressed zip file

Syntax:

zip archive.zip file

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ zip files.zip file1.txt file2.txt
adding: file1.txt (stored 0%)
adding: file2.txt (stored 0%)
```

Command 5: unzip - extract zip archive file

Syntax:

unzip archive.zip

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ unzip files.zip
Archive:  files.zip
replace file1.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
extracting: file1.txt
replace file2.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
extracting: file2.txt
```

Command 6: xz - compress files using xz

Syntax:

xz filename

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ xz file1.txt
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ ls
file1.txt.xz  file2.txt  files.zip  log.txt.gz
```

Command 7: unxz - decompress xz compressed file

Syntax:

unxz filename.xz

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ unxz file1.txt.xz
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ ls
file1.txt  file2.txt  files.zip  log.txt.gz
```

R1(5)	R2(5)	R3(5)	R4(5)	Total (20)

RESULT:

Thus, the archiving and compression utilities using tar, gzip, gunzip, zip, unzip, and xz were successfully executed in Linux and the expected outputs were obtained.

Exp : 7
Date:

SHELL SCRIPTING BASICS WRITING SIMPLE SHELL SCRIPTS

AIM:

To practice and understand Linux shell scripting, basic shell commands, and simple scripts effectively and efficiently in a structured environment.

PROCEDURE:

1. Open the Linux terminal on the system.
2. Create a new script file using a text editor (nano or vi) with .sh extension.
3. Write a simple “Hello World” program using the echo command.
4. Save the script file and exit the editor.
5. Provide execution permission using `chmod +x filename.sh`.
6. Execute the script using `./filename.sh` and observe the output.
7. Add comments in the script using `#` to describe each step.
8. Declare and use variables to store values and display them using echo.
9. Use the read command to accept user input from the keyboard.
10. Perform basic arithmetic operations using `expr` and `let`.
11. Modify and re-run the script to verify different outputs and understand functionality.

LINUX COMMANDS :

Command 1:echo – display the output

Syntax:echo “hello World”

Output:

```
rithik@DESKTOP-80QDGT4:~$ echo Hello World
Hello World
rithik@DESKTOP-80QDGT4:~$ |
```

2. Command 2:script execution.

Syntax:./<fileName>

```
GNU nano 7.2
echo hello world
```

Output :

```
rithik@DESKTOP-80QDGT4:~$ ./linux7.sh
hello world
rithik@DESKTOP-80QDGT4:~$ |
```

3. Command 3: comments on the program

Syntax: #comments

```
GNU nano 7.2
echo hello world
#This is a comment
```

Output:

```
rithik@DESKTOP-80QDGT4:~$ ./linux7.sh
hello world
rithik@DESKTOP-80QDGT4:~$ |
```

4. Command 4 :variable and storing a variable using let

Syntax: let var=val

```
GNU nano 7.2
#Comment

let a=5
let b=3
let sum=a+b

echo "Sum = $sum"
```

Output:

```
rithik@DESKTOP-80QDGT4:~$ ./linux7.sh
Sum = 8
rithik@DESKTOP-80QDGT4:~$ |
```

5. Command 5 :read variable

Syntax: read var

```
GNU nano 7.2
#Comment

echo "Enter your name:"
read name

echo "Hello $name"
```

Output :

```
rithik@DESKTOP-80QDGT4:~$ ./linux7.sh
Enter your name:
rithik
Hello rithik
rithik@DESKTOP-80QDGT4:~$ |
```

6. Command 6 : expr – expression

Syntax: expr

```
GNU nano 7.2
#Comment
a=10
b=5

sum=`expr $a + $b`
echo "Sum = $sum"|
```

Output:

```
rithik@DESKTOP-80QDGT4:~$ ./linux7.sh
Sum = 15
rithik@DESKTOP-80QDGT4:~$ |
```

R1(5)	R2(5)	R3(5)	R4(5)	Total (20)

RESULT:

The basic shell commands and simple shell scripts such as Hello World, variables, user input, and arithmetic operations were successfully executed. The outputs were verified, and the functionality of shell scripting concepts was understood.

Exp : 8

Date:

CONDITIONAL STATEMENTS IN SHELL

AIM:

To practice shell scripting and conditional statements in shell effectively and efficiently.

PROCEDURE:

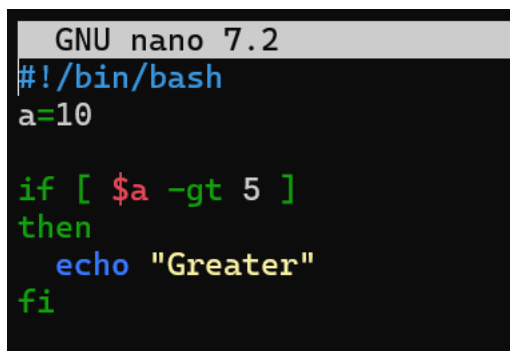
1. Log in to the Linux system and open the terminal.
2. Explore the Shell Scripting Basics and Writing Simple Shell Scripts.
3. Use the shell scripting commands to write conditional statements.
4. Write and execute a script using the if statement to test a condition.
5. Write and execute a script using the if-else statement for two-way branching.
6. Write and execute a script using the if-elif-else statement for multi-way branching.
7. Use the test command and integer/string comparison operators in shell scripts.
8. Verify the output of each script and document the results.

Linux Commands.

1. Command 1: if - check condition and execute block

Syntax:

```
if [ condition ]  
then  
  commands  
fi
```



```
GNU nano 7.2  
#!/bin/bash  
a=10  
  
if [ $a -gt 5 ]  
then  
  echo "Greater"  
fi
```

Output:

```
rithik@DESKTOP-80QDGT4:~$ ./linux7.sh
Greater
rithik@DESKTOP-80QDGT4:~$ |
```

2. Command 2: if else. - choose between two conditions

Syntax: if [condition] then <statement> else,

```
GNU nano 7.2
#!/bin/bash
a=3

if [ $a -gt 5 ]
then
    echo "Greater"
else
    echo "Smaller"
fi|
```

Output:

```
rithik@DESKTOP-80QDGT4:~$ ./linux7.sh
Smaller
rithik@DESKTOP-80QDGT4:~$ nano linux7.sh
```

3. Command 3: if-elif-else - check multiple conditions stepwise

Syntax:

```
if [ condition ]
then
    commands
elif [ condition ]
then
    commands
else
    commands
fi
```

```
#!/bin/bash
a=5

if [ $a -gt 5 ]
then
    echo "Greater"
elif [ $a -eq 5 ]
then
    echo "Equal"
else
    echo "Smaller"
fi
```

Output:

```
rithik@DESKTOP-80QDGT4:~$ ./linux7.sh
Equal
rithik@DESKTOP-80QDGT4:~$ |
```

4. Command 4: test - check multiple conditions stepwise

Syntax: test condition

```
#!/bin/bash
a=10

if test $a -gt 5
then
    echo "True"
fi
```

Output:

```
rithik@DESKTOP-80QDGT4:~$ ./linux7.sh
True
rithik@DESKTOP-80QDGT4:~$ |
```

5. Command 5: integer comparison - compare numeric values in shell

Syntax: [\$a -gt \$b]

```
GNU nano 7.2
#!/bin/bash
a=10
b=5

if [ $a -gt $b ]
then
    echo "a is greater"
fi
```

Output:

```
rithik@DESKTOP-80QDGT4:~$ ./linux7.sh
a is greater
rithik@DESKTOP-80QDGT4:~$ |
```

6. Command 6: string comparison - compare string values in shell

Syntax: ["\$a" = "\$b"]

```
GNU nano 7.2
#!/bin/bash
a="hi"
b="hi"

if [ "$a" = "$b" ]
then
    echo "Equal"
fi
```

Output:

```
rithik@DESKTOP-80QDGT4:~$ ./linux7.sh
Equal
rithik@DESKTOP-80QDGT4:~$ |
```

7. Command 7: File comparison - check file properties existence/type

Syntax: [-f filename]

```

GNU nano 7.2
#!/bin/bash

if [ -f file.txt ]
then
    echo "File exists"
else
    echo "File not found"
fi

```

Output:

```

rithik@DESKTOP-80QDGT4:~$ ./linux7.sh
File not found
rithik@DESKTOP-80QDGT4:~$ |

```

R1(5)	R2(5)	R3(5)	R4(5)	Total (20)

RESULT:

The shell scripting commands the Conditional statements are executed successfully.

Exp : 9

Date:

LOOPING CONSTRUCTS IN SHELL

AIM:

To study and implement looping statements (for, while, until) and loop control commands (break, continue) in Linux shell scripting.

PROCEDURE:

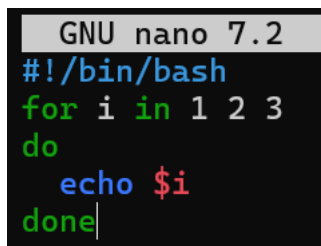
1. Open the Linux terminal.
2. Create a new shell script file using nano editor.
3. Write a program using for loop to iterate values.
4. Write a program using while loop with condition.
5. Write a program using until loop for execution.
6. Implement break and continue statements in loops.
7. Save, give permission using chmod +x, and execute the script.

LINUX COMMANDS

Command 1: for - repeat loop for list values

Syntax:

```
for var in list
do
  commands
done
```

A screenshot of a terminal window with a dark background. The title bar at the top says "GNU nano 7.2". The text inside the terminal shows a shell script being edited: the first line is "#!/bin/bash" in blue, followed by "for i in 1 2 3" in green, "do" in green, "echo \$i" in blue, and "done" in green. A cursor is visible at the end of the "done" line.

```
GNU nano 7.2
#!/bin/bash
for i in 1 2 3
do
  echo $i
done
```

Output :

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ nano linux7.sh
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ ./linux7.sh
1
2
3
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ |
```

Command 2: while - repeat while condition is true

Syntax :

while [condition]

do

 commands

Done

```
GNU nano 7.2
#!/bin/bash
i=1
while [ $i -le 3 ]
do
    echo $i
    let i++
done|
```

Output :

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ nano linux7.sh
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ ./linux7.sh
1
2
3
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ |
```

Command 3 : until - repeat until condition becomes true

Syntax:

until [condition]

do

 commands

done

```
GNU nano 7.2
#!/bin/bash
i=1
until [ $i -gt 5 ]
do
    echo $i
    let i++
done
```

Output :

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ ./linux7.sh
1
2
3
4
5
```

Command 4 : Break - exit loop immediately

Syntax: break

```
GNU nano 7.2
#!/bin/bash
for i in 1 2 3 4
do
    if [ $i -eq 3 ]
    then
        break
    fi
    echo $i
done|
```

Output :

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ ./linux7.sh
1
2
```

Command 5 : Continue - skip current iteration

Syntax : continue

```

GNU nano 7.2
#!/bin/bash
for i in 1 2 3 4
do
    if [ $i -eq 3 ]
    then
        continue
    fi
    echo $i
done

```

Output :

```

rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ ./linux7.sh
1
2
4

```

R1(5)	R2(5)	R3(5)	R4(5)	Total (20)

RESULT:

Thus, the looping statements (for, while, until) and loop control commands (break, continue) were successfully executed in Linux shell scripting, and the expected outputs were obtained.

Exp : 10

Date:

USER-DEFINED FUNCTIONS IN SHELL SCRIPTING

AIM :

To study and implement user-defined functions in shell scripting with function declaration, function calling, passing arguments, and return values.

PROCEDURE :

1. Open the Linux terminal.
2. Create a shell script file using nano.
3. Declare functions using function syntax.
4. Call the functions inside the script.
5. Pass arguments using \$1, \$2, \$@, and \$#.
6. Use return statement in functions.
7. Execute the script and verify the output.

LINUX COMMANDS :

Command 1: Function Declaration and calling

Use: define reusable block of code

Syntax:

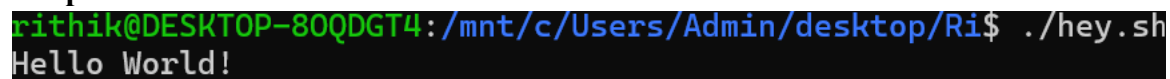
```
function_name() {  
  commands  
}
```

```
function_name
```



```
GNU nano 7.2  
#!/bin/bash  
  
greet() {  
  echo "Hello World!"  
}  
  
greet
```

Output:



```
rithik@DESKTOP-80QDGT4: /mnt/c/Users/Admin/desktop/Ri$ ./hey.sh  
Hello World!
```

Command 2: Function with Arguments

Use:

1. First Argument = \$1
2. Second Argument = \$2
3. All Arguments = \$@
4. Total Arguments = \$#

Syntax:

Passing Arguments, \$1, \$2, \$@, \$#

```
GNU nano 7.2
#!/bin/bash

show() {
echo "First Argument = $1"
echo "Second Argument = $2"
echo "All Arguments = $@"
echo "Total Arguments = $#"
}

show Hello World
```

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ ./hey.sh
First Argument = Hello
Second Argument = World
All Arguments = Hello World
Total Arguments = 2
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$
```

Command 3: Return Value in Function

Use: return status from function

Syntax: return value

```
GNU nano 7.2
#!/bin/bash

check() {
if [ $1 -gt 10 ]
then
echo "Number is Greater"
return 1
else
echo "Number is Smaller"
return 0
fi
}

check 5
echo "Return Value = $?"
```

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ ./hey.sh
Number is Smaller
Return Value = 0
```

Command 4: local Variable

Use: determine local Variable.

Syntax: local Variable.

```
GNU nano 7.2
#!/bin/bash

calculator() {
  local a=$1
  local b=$2
  local result=$((a + b))

  echo "First Number = $a"
  echo "Second Number = $b"
  echo "Result = $result"
}

calculator 4 6
```

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ ./hey.sh
First Number = 4
Second Number = 6
Result = 10
```

R1(5)	R2(5)	R3(5)	R4(5)	Total (20)

RESULT:

Thus, the user-defined functions in shell scripting were successfully implemented using function declaration, function calling, arguments, return values, and local variables.

Exp : 11
Date:

Command Line Arguments and Script Automation

AIM

To study command line arguments and script automation in shell scripting using special variables and loops.

PROCEDURE

1. Open the Linux terminal.
2. Create a shell script file using nano.
3. Write a program using special variables like \$0, \$1, \$2, \$@, \$*, and \$#.
4. Pass arguments while running the script.
5. Use loop statements to print all arguments.
6. Execute the shell script in terminal.
7. Verify the output obtained from the script.

LINUX COMMANDS

Command 1: Special Variables

Use: access command line arguments

Syntax:

\$0

\$1

\$2

\$@

\$*

\$#

```
GNU nano 7.2
#!/bin/bash

echo "Program Name = $0"
echo "First Argument = $1"
echo "Second Argument = $2"
echo "Total Arguments = $#"
```



```
echo "Using \@ = @"
echo "Using \* = "*>
```

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ ./hey.sh Cat Dog Fish
Program Name = ./hey.sh
First Argument = Cat
Second Argument = Dog
Total Arguments = 3
Using $@ = Cat Dog Fish
Using $* = Cat Dog Fish
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ |
```

Command 2: Looping Through Arguments

Use: print arguments one by one

Syntax:

for variable in "\$@"

do

commands

done

```
GNU nano 7.2
#!/bin/bash

for item in "$@"
do
echo "Value = $item"
done|
```

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin$ ./hey.sh Rose Lily Lotus Tulip
Value = Rose
Value = Lily
Value = Lotus
Value = Tulip
```

Command 3: Simple Backup Automation

Use: automate copying of files

Syntax: cp source destination

```
GNU nano 7.2
#!/bin/bash

cp data.txt data_backup.txt
echo "Backup Completed"
```

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ ./hey.sh
Backup Completed
```

Command 4: Simple Log Monitoring

Use: display contents of log file

Syntax: cat filename

```
GNU nano 7.2
#!/bin/bash

cat server.log
```

Output:

```
rithik@DESKTOP-80QDGT4:/mnt/c/Users/Admin/desktop/Ri$ ./hey.sh
[2026-05-10 08:15:21] INFO      Server started on port 8080
[2026-05-10 08:15:25] INFO      Client connected: 192.168.1.12
[2026-05-10 08:16:02] WARNING  High memory usage detected (78%)
[2026-05-10 08:16:15] INFO      User login successful: admin
[2026-05-10 08:17:08] ERROR    Database connection timeout
[2026-05-10 08:17:30] INFO      Retrying database connection...
[2026-05-10 08:17:35] INFO      Database connection restored
[2026-05-10 08:18:10] INFO      File uploaded: report.pdf
[2026-05-10 08:18:42] WARNING  Disk space low: 12% remaining
[2026-05-10 08:19:05] INFO      Client disconnected: 192.168.1.12
[2026-05-10 08:20:00] INFO      Scheduled backup completed successfully
```

R1(5)	R2(5)	R3(5)	R4(5)	Total (20)

RESULT:

Thus, the command line arguments and script automation using special variables and loops were successfully implemented in shell scripting.